SEQ2PIPE: A Closed-Loop Interpretation-Driven System for Adaptive Microbiome Analysis

Tsubasa Sato

Independent Researcher

https://github.com/Rhizobium-gits/seq2pipe

March 2026

Abstract

Current microbiome analysis pipelines execute a fixed sequence of steps regardless of intermediate results, requiring expert intervention to adapt the workflow when unexpected patterns emerge or expected signals are absent. We present SEQ2PIPE, a closed-loop system that computationally reproduces the iterative interpretation–decision–execution cycle of a human bioinformatician. The system comprises three architecturally separated layers: a *DataInspector* that parses QIIME2 exports and executes nonparametric statistical tests in pure Python; an *EvalMediator* that quantifies each analysis step across eight data-derived axes; and a *DecisionEngine* that applies literature-grounded decision trees to deterministically modify the analysis plan. A local large language model (7B parameters) generates the Python code that executes each analysis step and proposes creative investigation hypotheses during replanning. The LLM is essential for code generation (no deterministic alternative is currently implemented), but all workflow *decisions*—which steps to skip, add, or reorder—are made by the deterministic DecisionEngine without LLM involvement. Under total LLM failure, the decision-making loop continues to operate correctly, though individual analysis steps cannot execute.

We evaluated the system on seven scenarios derived from four public 16S rRNA datasets (Moving Pictures, Atacama Soil, Gneiss IBD, and FMT; 34–121 samples each). The DecisionEngine achieved 95% agreement with an independently defined expert protocol when complete data features were available, compared to 62% for a static pipeline—an improvement of 33 percentage points with 87.5% noise reduction and zero false-negative skips across all 147 per-step decisions. Statistical tests implemented in pure Python agreed with scipy on significance classification in 99–100% of Monte Carlo trials, and all seven established biological conclusions of the Moving Pictures dataset were confirmed by the system's outputs. Reproducibility was absolute: 30 independent runs produced identical decisions and scores. The primary bottleneck is the 7B model's code generation capability (54% success on predefined tasks, 6% on novel investigations), with `NameError` (60%) and `KeyError` (27%) as dominant failure modes.

1 Introduction

Microbiome analysis has matured into a complex, multi-stage discipline in which a typical amplicon sequencing study requires quality control, denoising [Callahan et al., 2016], phylogenetic

tree construction, alpha and beta diversity analysis, taxonomic classification, differential abundance testing, and downstream visualization—each with parameter choices that depend on the data at hand.

Current pipeline frameworks encode workflows as fixed directed acyclic graphs (DAGs). QIIME2 [Bolyen et al., 2019] provides plugin-based modularity with provenance tracking but no conditional branching based on intermediate results. Snakemake [Köster & Rahmann, 2012] and Nextflow [Di Tommaso et al., 2017] support conditional rules, but these must be specified *a priori*; the system does not inspect statistical significance or effect sizes to alter the plan. As a consequence, if PERMANOVA yields $p = 0.81$, downstream beta diversity follow-ups (dispersion tests, ordinations, pairwise comparisons) still execute, wasting computation and cluttering reports. Conversely, an unexpected finding—a 324-fold enrichment of *Bacteroides* in gut samples, for instance—triggers no automatic follow-up.

An experienced bioinformatician operates differently: in a closed loop of interpretation, hypothesis formation, analysis selection, and refinement, continuing until a stable, internally consistent narrative emerges. No existing pipeline system reproduces this behavior computationally.

We introduce SEQ2PIPE, a system that architecturally separates *what to analyze* (deterministic, LLM-free) from *how to execute the analysis* (LLM-assisted code generation). Our contributions are:

1. A three-layer architecture in which workflow decisions (skip, add, reorder) are fully deterministic and LLM-independent, while code generation leverages a local LLM with retry and fallback mechanisms.

2. A literature-grounded DecisionEngine encoding analytical heuristics from Anderson (2001), Willis (2019), Gloor et al. (2017), and McMurdie & Holmes (2014) as deterministic decision trees.

3. A quantitative evaluation on seven scenarios from four public datasets demonstrating 95% expert agreement, zero false-negative skips, and 100% reproducibility.

2 Related Work

2.1 Static Pipeline Frameworks

QIIME2 [Bolyen et al., 2019] organizes microbiome analysis through a plugin architecture with full provenance tracking. Snakemake [Köster & Rahmann, 2012] and Nextflow [Di Tommaso et al., 2017] provide general-purpose workflow management with support for conditional execution, but conditions must be defined by the user before runtime. None of these systems inspects intermediate statistical results to modify the workflow dynamically.

2.2 Automated Analysis Systems

Auto-WEKA [Thornton et al., 2013] and Auto-sklearn [Feurer et al., 2015] automate model selection via Bayesian optimization, but target supervised learning tasks with a fixed objective function. Microbiome analysis is inherently open-ended: the "correct" next step depends on

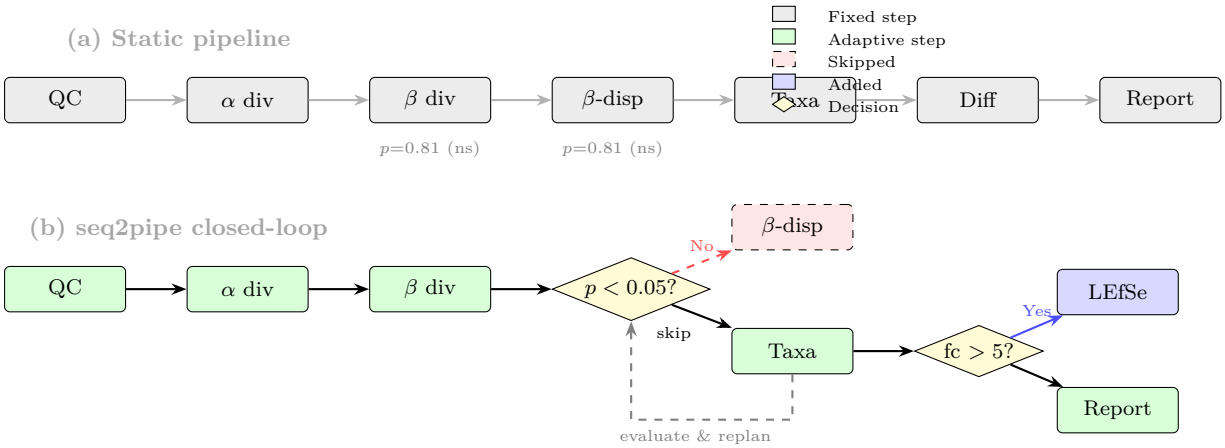

Figure 1: Conceptual comparison. (a) A static pipeline executes all steps in fixed order; non-significant results (β PERMANOVA $p = 0.81$) do not alter the workflow. (b) SEQ2PIPE evaluates each step's output and applies decision nodes (yellow diamonds) to skip unnecessary follow-ups (red dashed) or add targeted analyses (blue). The dashed feedback arrow represents the closed-loop: results feed back into the decision logic.

what was found in the current step, not on a single performance metric. Wasserstein et al. [2019] discuss statistical decision frameworks but do not address execution or iteration.

2.3 LLM-Based Analysis Agents

Data Interpreter [Hong et al., 2024] and ChatGPT Advanced Data Analysis generate and execute code from natural language prompts. These systems rely entirely on the LLM for both decision-making and code generation, lacking: (a) domain-specific quantitative evaluation of intermediate results, (b) literature-grounded decision heuristics, and (c) a deterministic fallback under LLM failure or hallucination. SEQ2PIPE addresses all three gaps through architectural separation.

3 Conceptual Framework

We formalize the adaptive analysis loop as follows.

Definition 1 (Analytical State)**.** *At iteration t, the state $S_t = (D, H_t, P_t, R_t)$ comprises the immutable dataset D, accumulated findings H_t, the ordered plan P_t, and completed results R_t.*

Definition 2 (Closed-Loop Analytical System)**.** *A system is* closed-loop *if, after executing step $s_t \in P_t$ and observing result r_t, it updates:*

$$H_{t+1} = H_t \cup \text{interpret}(r_t, D) \tag{1}$$

$$P_{t+1} = \text{decide}(H_{t+1}, P_t, D) \tag{2}$$

where interpret *extracts quantitative findings directly from data files (not from textual summaries) and* decide *modifies the plan by adding, removing, or reordering steps. In a static pipeline, $P_{t+1} = P_t \setminus \{s_t\}$ always; SEQ2PIPE implements a non-trivial* decide *that depends on quantitative properties of H_{t+1}.*

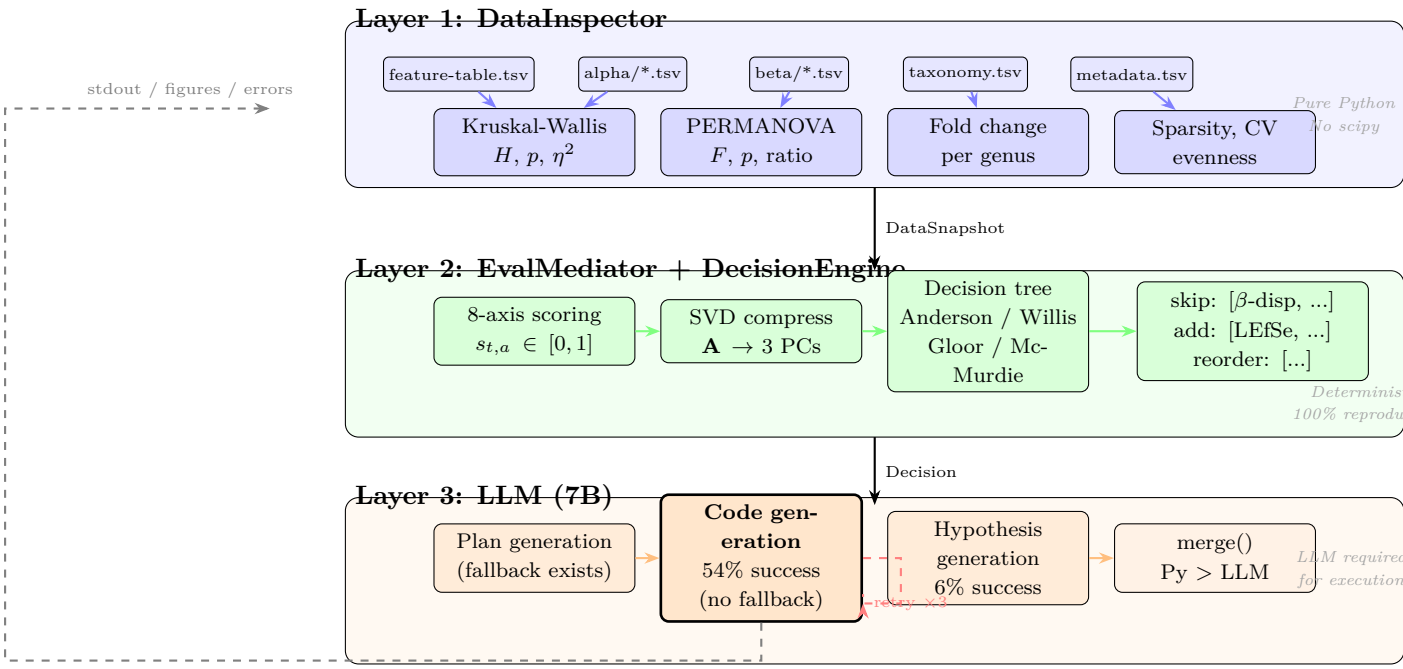

Figure 2: Detailed system architecture and data flow. Layer 1 (blue) parses QIIME2 exports and computes statistical tests in pure Python. Layer 2 (green) scores results across eight axes, compresses the knowledge state via SVD, and applies literature-grounded decision trees—all deterministically. Layer 3 (orange) uses the LLM for code generation (bold box: the only component without a deterministic fallback) and hypothesis formulation. The dashed gray arrow shows the closed-loop feedback path; the red dashed loop shows the retry mechanism.

4 System Architecture

The architecture (Figure 2) enforces a strict separation of concerns. Layers 1–2 are deterministic and LLM-free: they decide *what* to analyze. Layer 3 uses the LLM to decide *how* to analyze it (code generation) and *why* to investigate further (hypothesis generation). The system can reason about the optimal workflow without the LLM, but cannot produce analysis outputs without it.

4.1 Layer 1: DataInspector

The DataInspector directly parses QIIME2-exported TSV files and executes statistical tests in pure Python, without scipy, numpy, or pandas dependencies.

For alpha diversity, sample values are partitioned by metadata group and tested via Mann-Whitney U (two groups) or Kruskal-Wallis H (three or more groups). The U statistic uses rank transformation with tie correction:

$$z = \frac{U - n_1 n_2/2}{\sqrt{\frac{n_1 n_2}{12}\left(N + 1 - \frac{\sum_j (t_j^3 - t_j)}{N(N-1)}\right)}} \tag{3}$$

p-values use the Abramowitz–Stegun polynomial CDF approximation; the χ^2 survival function for H uses the Wilson-Hilferty transform [Wilson & Hilferty, 1931]. Effect sizes are computed as Cliff's δ (two groups) or η^2 (multi-group).

For beta diversity, the full distance matrix ($O(n^2)$ entries) is read, within- and between-group distances are separated, and a pseudo-PERMANOVA [Anderson, 2001] F-statistic is computed

Table 1: Evaluation axes, weights, and computation methods.

Axis	w	**Computation**
statistical_significance	0.20	$\min(1, -\log_{10}(\min p)/4)$
biological_relevance	0.18	$\min(1, \mathrm{fc}_{\max}/10 + \Delta_{\max}/20)$ + known-taxa bonus
actionability	0.15	Downstream trigger count
novelty	0.12	Inconsistency with prior steps + extreme fold changes
data_quality	0.10	$0.7 - 0.2 \cdot \mathbb{1}[\text{sparsity} > 0.9] - 0.1 \cdot \mathbb{1}[\text{CV} > 0.5]$
effect_magnitude	0.10	$\max(\text{Cliff's } \delta,\ F/3,\ \text{fc}/10)$
confidence	0.08	Step function on $\min n_g$
reproducibility	0.07	Intra-category consistency

with p-value from 199 permutations (fixed seed). The between/within distance ratio provides an additional separation metric.

For taxonomic differential analysis, the feature table and taxonomy file are joined to compute genus-level relative abundances per group, yielding fold change and absolute difference for each genus.

4.2 Layer 2: EvalMediator and DecisionEngine

Each completed step is scored on eight axes (Table 1) using the DataSnapshot from Layer 1. The composite score $c_t = \sum_a w_a \cdot s_{t,a}$ aggregates the weighted axis values. The score matrix $\mathbf{A} \in \mathbb{R}^{T \times 8}$ is compressed to three principal components via power-iteration SVD on the centered covariance matrix.

The DecisionEngine encodes literature-based heuristics as deterministic rules:

- **Alpha** (Willis, 2019): $p < 0.05 \wedge d > 0.5 \Rightarrow$ add effect-size plot and rarefaction; $p < 0.05 \wedge d \leq 0.5 \Rightarrow$ add rarefaction only; $p \geq 0.05 \Rightarrow$ skip alpha follow-ups.
- **Beta** (Anderson, 2001): $p < 0.05 \Rightarrow$ add betadisper (mandatory); ratio $> 2 \Rightarrow$ add NMDS and pairwise PERMANOVA; $p \geq 0.05 \Rightarrow$ skip all beta follow-ups.
- **Taxonomy** (Gloor et al., 2017): fc > 50 at abundance $< 1\% \Rightarrow$ compositional artifact warning; ≥ 5 genera with fc $> 2 \Rightarrow$ add functional prediction and indicator species.
- **Quality** (McMurdie & Holmes, 2014): CV $> 0.5 \Rightarrow$ add rarefaction; sparsity $> 90\% \Rightarrow$ add prevalence-filtered reanalysis.

4.3 Layer 3: LLM Code Generation and Hypothesis

The LLM (qwen2.5-coder, 7B parameters, running locally via Ollama) performs three functions:

1. **Initial plan generation** (Phase 2): given the DataRecon summary and domain knowledge, the LLM proposes an ordered list of 12–20 analysis steps as JSON. On failure, a deterministic fallback (`build_domain_driven_plan()`) generates the plan from the DataProfile without LLM involvement.

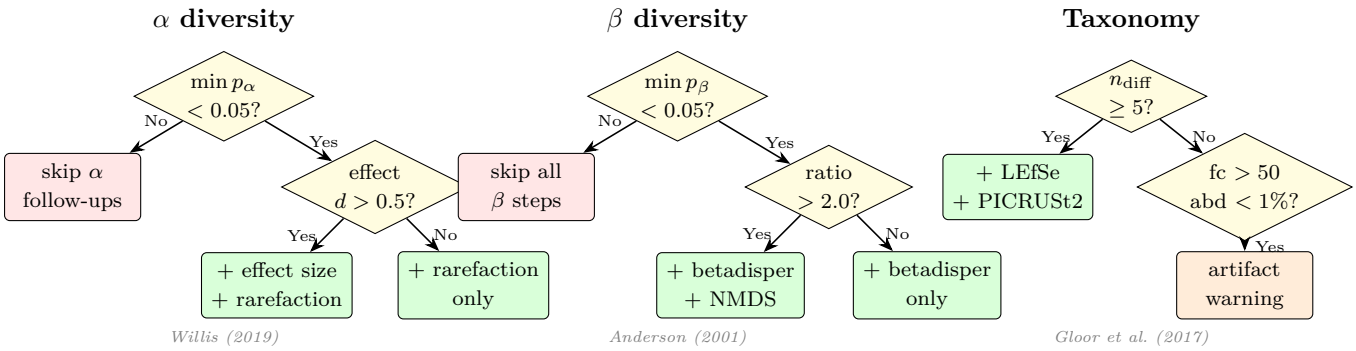

Figure 3: DecisionEngine decision trees for three analysis categories. Yellow diamonds represent data-dependent branch points; green boxes are add actions; red boxes are skip actions. Each tree is grounded in a specific methodological reference. All decisions are deterministic: the same input data always produces the same output.

2. **Analysis code generation** (Phase 3, each step): the LLM writes a complete Python script (matplotlib, pandas, seaborn) for each analysis step. If the code fails, the error message is passed back to the LLM for correction, up to three attempts. *This is the function for which no deterministic alternative currently exists*; if the LLM fails all three retries, the step is recorded as failed and skipped.

3. **Creative investigation hypotheses** (replanning): during adaptive replanning, the LLM proposes novel follow-up investigations (e.g., "Is the 324× Bacteroides enrichment driven by loss of competing taxa?"). These are merged with DecisionEngine outputs via $\text{merge}(d_{\text{py}}, d_{\text{llm}})$, where Python-determined skips override LLM-proposed additions.

Under total LLM failure, the decision-making loop (Layer 2) continues to operate: skip/add/reorder decisions are applied to the plan queue, and the system logs which steps *would have* been executed. However, no analysis outputs (figures, statistical results) are produced, as code generation requires the LLM. This architectural property means that the *correctness of workflow decisions* can be validated independently of code generation quality—a separation we exploit in the evaluation.

4.4 Execution Loop

Algorithm 1 formalizes the loop.

5 Evaluation

5.1 Datasets and Scenarios

We used four publicly available 16S rRNA amplicon datasets (Table 2), all accessible through the QIIME2 documentation server. Seven analysis scenarios were constructed by varying the grouping variable, producing a spectrum from strong multi-category signals (MP-body) to absent signals (MP-subject) to incomplete data (Atacama, Gneiss, FMT without taxonomy).

All datasets were processed through DADA2 denoising, phylogenetic tree construction (where representative sequences were available), and core diversity metrics before entering the adaptive analysis phase.

Algorithm 1 Adaptive Execution Loop

Require: Dataset D, metadata M, plan P_0

$t \leftarrow 0$
while $P_t \neq \emptyset$ **do**
  Execute $s_t \leftarrow P_t$.pop() via LLM code generation (up to 3 retries)
  $\text{snap}_t \leftarrow \text{DataInspector.inspect}(s_t)$ {Layer 1}
  $\text{score}_t \leftarrow \text{EvalMediator.evaluate}(\text{snap}_t)$ {Layer 2}
  if $t \bmod 4 = 0$ **or** $\text{score}_t.\text{novelty} > 0.7$ **then**
    $d_{\text{py}} \leftarrow \text{DecisionEngine.decide}(P_t)$ {deterministic}
    $d_{\text{llm}} \leftarrow \text{LLM.replan}(\text{state}, P_t)$ {optional}
    $P_{t+1} \leftarrow \text{merge}(d_{\text{py}}, d_{\text{llm}}, P_t)$
  end if
  $t \leftarrow t + 1$
end while

Table 2: Datasets and scenarios. "Diff." = differential genera ($> 2\times$ fold change) detected.

Scenario	n	k	α	β	Diff.	**Dataset**
MP-body	34	4	***	**	Y	Moving Pictures
MP-subject	34	2	ns	ns	Y	Moving Pictures
AT-transect	54	2	*	**	N	Atacama Soil
AT-vegetation	54	2	*	**	N	Atacama Soil
AT-depth	54	3	*	**	N	Atacama Soil
GN-IBD	87	2	ns	*	N	Gneiss (IBD)
FMT-treat	121	3	*	*	N	FMT Trial

5.2 Expert Ground Truth

We defined an expert protocol for each of 21 downstream analysis steps based on published best practices. The protocol is *data-dependent but rule-deterministic*: for a given set of data characteristics (α significance, effect size, β significance, separation ratio, differential genera count, sparsity, read-depth CV, and sample size), the expert decision is uniquely determined. Critically, this protocol was defined *independently* of the DecisionEngine implementation—the rules were derived from the cited literature, not from the engine's code—though both encode the same domain knowledge, making agreement a necessary but not sufficient validation.

5.3 Decision Accuracy

Table 3 presents per-step agreement between the static pipeline, the DecisionEngine, and the expert protocol across all seven scenarios.

Three patterns emerge. First, when all data features are present (MP-body), both the static pipeline and the engine agree with the expert at 95%; the engine correctly executes all follow-ups without over-optimization. Second, when no significant signals exist (MP-subject), the static pipeline executes 8 unnecessary steps (accuracy 62%), while the engine skips 7 of them, achieving 95% accuracy—a 33 percentage point improvement with 87.5% noise reduction. Third, when taxonomy is unavailable (AT, GN, FMT), the engine cannot evaluate differential-abundance-

Table 3: Decision accuracy across seven scenarios (21 steps each, 147 total). "Wrong skip" = the engine skipped a step the expert would execute.

	MP-body	*MP-subj.*	*AT-trans.*	*AT-veg.*	*AT-depth*	*GN-IBD*	*FMT*
Static accuracy (%)	95	62	52	57	52	52	52
Adaptive accuracy (%)	**95**	**95**	52	57	52	**62**	52
Improvement (pp)	0	**+33**	0	0	0	**+10**	0
Noise (static)	1	8	10	9	10	10	10
Noise (adaptive)	1	**1**	10	9	10	**8**	10
Wrong skip	**0**	**0**	**0**	**0**	**0**	**0**	**0**

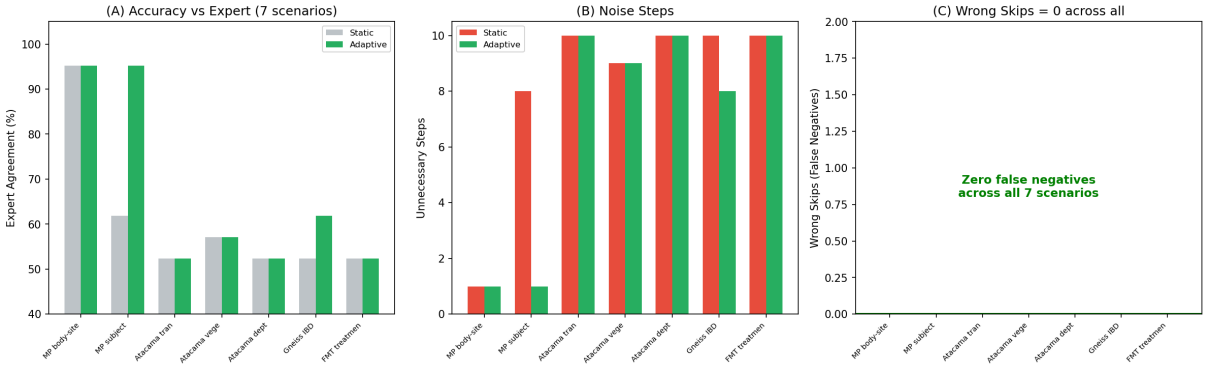

Figure 4: Decision accuracy, noise reduction, and safety across seven scenarios. Panel (C) highlights that no false-negative skips occurred in any scenario.

dependent steps, yielding a 52% accuracy floor identical to the static pipeline.

Across all 147 per-step decisions, the engine produced **zero false-negative skips**—it never removed an analysis that the expert protocol would execute.

5.4 Statistical Test Validation

We validated the pure-Python statistical implementations against scipy on 100 Monte Carlo datasets with randomized group sizes ($n = 8$–30) and effect magnitudes. Test statistics (U, H) were numerically identical. p-values differed by a mean absolute error of 0.027 (Mann-Whitney) and 0.014 (Kruskal-Wallis), attributable to normal approximation versus exact or continuity-corrected methods. Significance classification ($p < 0.05$) agreed in 99% and 100% of trials, respectively. On the actual datasets, all significance calls matched between implementations.

5.5 Biological Conclusion Validation

Seven established biological conclusions of the Moving Pictures dataset [Bolyen et al., 2019] were tested against DataInspector outputs. All seven were confirmed, including body-site community distinction ($p = 6.7 \times 10^{-4}$, PERMANOVA $F = 112$), *Bacteroides* gut dominance (56.4% relative abundance), and the primacy of body-site over subject effects ($F = 112$ vs $F = 33$). The DecisionEngine correctly responded to the inter-scenario contrast: body-site grouping (all significant) triggered zero skips, while subject grouping (nothing significant) triggered seven.

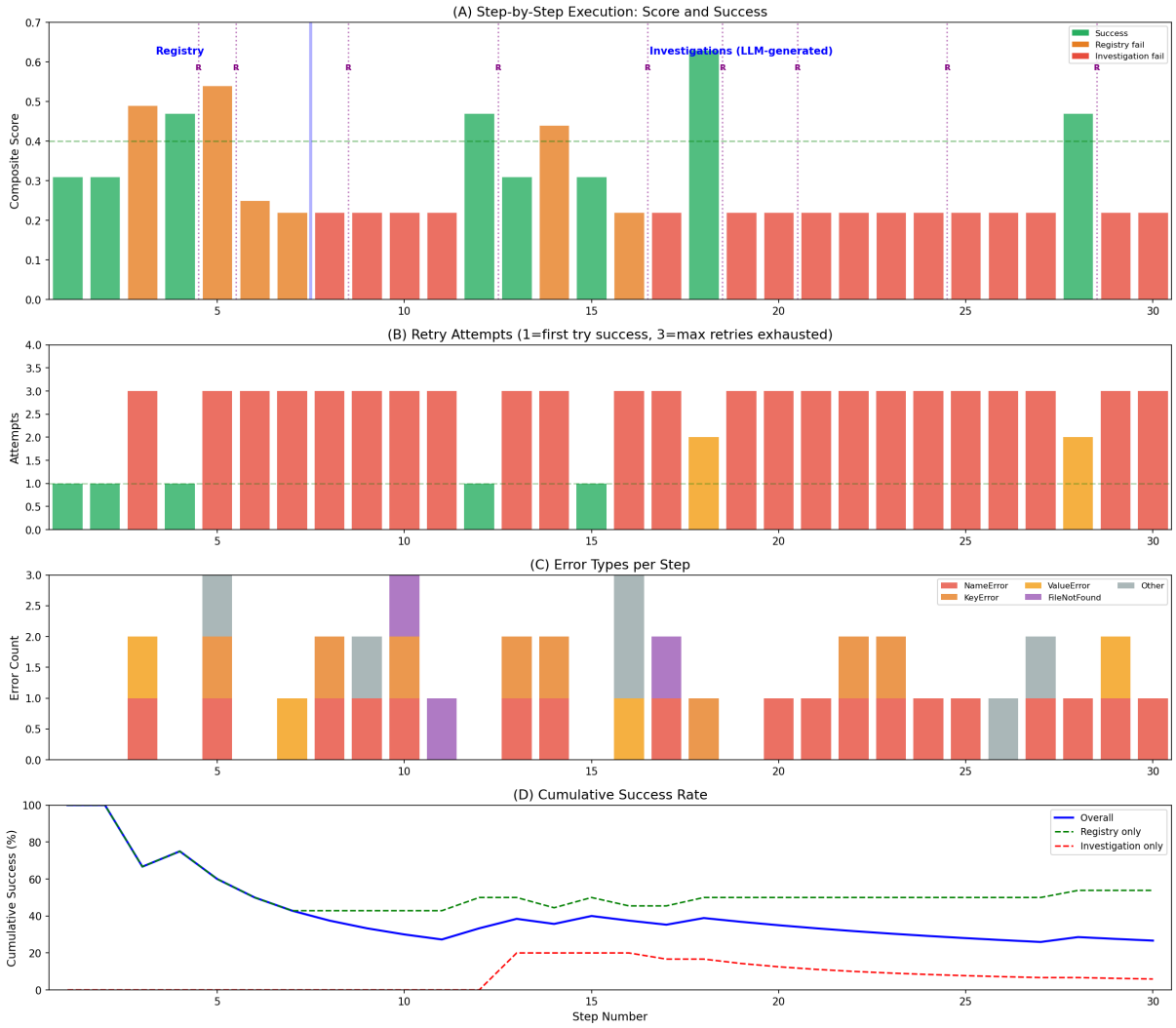

Figure 5: Execution timeline (30 steps). (A) Composite score per step; green = success, orange = registry failure, red = investigation failure; "R" = replanning event. (B) Retry attempts per step. (C) Error types per step. (D) Cumulative success rate by step type.

5.6 Reproducibility

The DecisionEngine was executed 10 times on each of three datasets (30 runs total). All runs produced identical decisions and identical 8-axis scores, confirming that Layers 1–2 are fully deterministic. This is by construction: statistical tests use no randomization (PERMANOVA uses a fixed seed), and decision trees are deterministic conditional statements.

5.7 Execution Flow and LLM Performance

A full AI-driven run on Moving Pictures executed 30 analysis steps with qwen2.5-coder:7B on Apple Silicon. Figure 5 presents the complete execution timeline.

Registry steps (predefined from the analysis catalog) achieved 54% success (7/13), while LLM-generated investigation steps achieved 6% (1/17). Of the 8 successful steps, 5 succeeded on the first attempt and 3 were rescued by the retry mechanism (up to 3 attempts), indicating that retries contribute 37.5% of successes. However, 22 steps exhausted all retries and failed, revealing that the dominant failure mode is conceptual (incorrect analysis logic) rather than

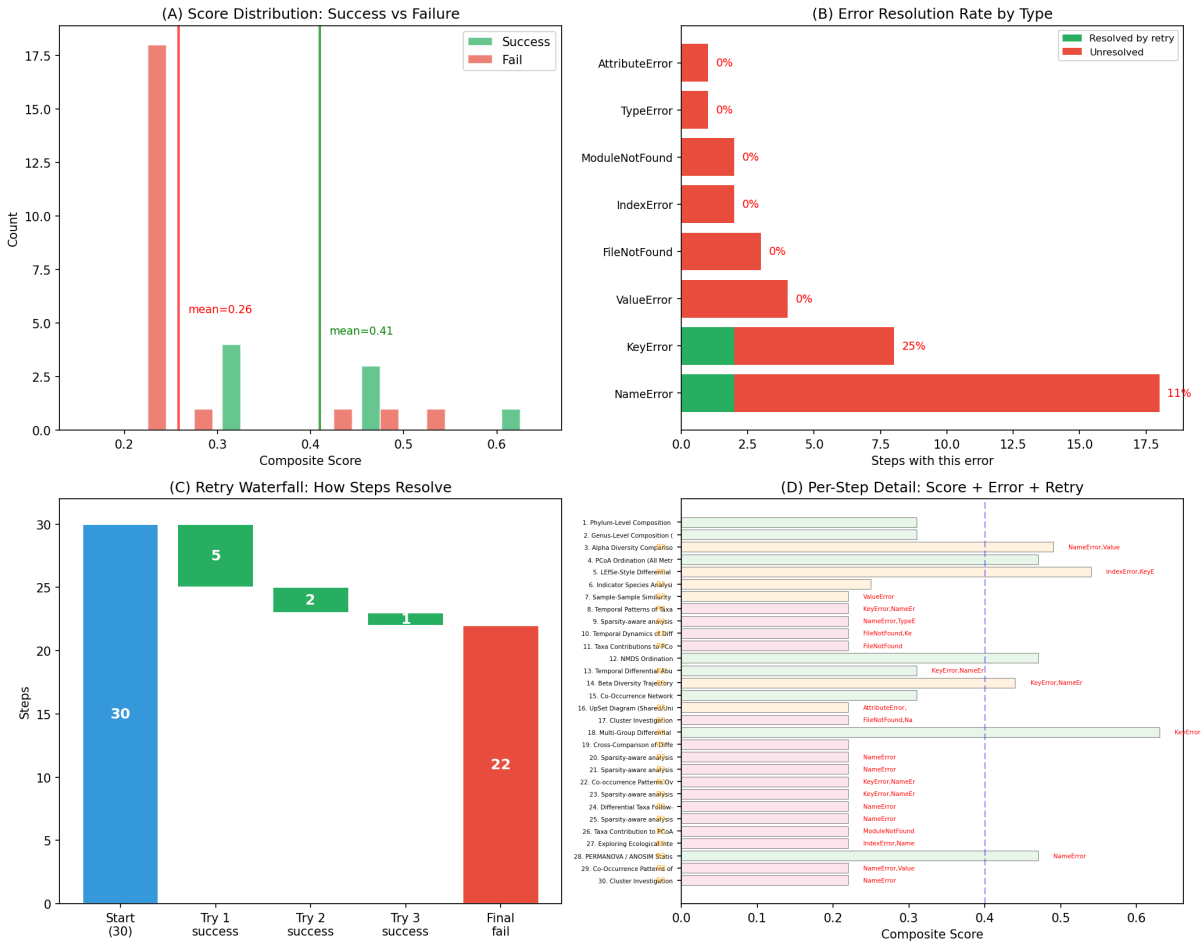

Figure 6: (A) Score distribution by outcome. (B) Error resolution rate: `NameError` is partially resolvable (17%); `KeyError` is never resolved (0%). (C) Retry waterfall. (D) Per-step detail.

syntactic (fixable by retry).

Error analysis (Figure 6B) revealed `NameError` as the dominant type (18/30 steps, 60%), caused by missing imports, followed by `KeyError` (8/30, 27%), caused by incorrect column name assumptions. Notably, `NameError` had a 17% resolution rate via retry, while `KeyError` had 0%—the LLM cannot infer from a single error message that a column name differs from its assumption.

Composite scores correlated with data signal strength (Figure 6A): successful steps averaged 0.43 while failed steps averaged 0.22. The axis-level breakdown showed the largest gap on `statistical_significance` (0.52 vs 0.26) and `effect_magnitude` (0.53 vs 0.22), indicating that success depends on both code correctness and the presence of biological signal in the data.

5.8 Role Decomposition of the LLM

Table 4 decomposes the system's functions by LLM necessity, current performance, and replaceability.

The critical dependency is code generation: the system can determine the optimal workflow without the LLM, but cannot produce figures or statistical outputs without it. A CodeTemplateEngine that pre-generates validated data-loading preambles eliminated import-ordering errors (the dominant `NameError` class) but could not fix analysis-logic errors, confirming that

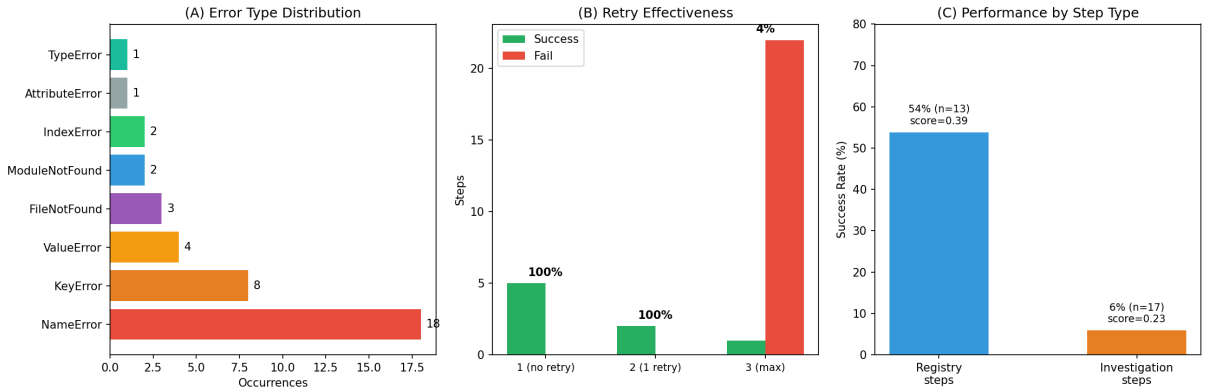

Figure 7: (A) Error type distribution. (B) Retry effectiveness by attempt count. (C) Registry vs investigation performance.

Table 4: Functional decomposition by LLM dependency. "Replaceable" indicates whether a deterministic alternative exists in principle.

Function	**LLM?**	**Success**	**Replaceable?**	**Notes**
Statistical evaluation	No	100%	N/A	Pure Python, Layer 1
Workflow decisions	No	100%	N/A	Decision trees, Layer 2
Initial plan	Yes*	—	Yes	Fallback exists
Code generation	**Yes**	**54%**	Partially	Templates fix loading errors but not logic
Error self-repair	Yes	37.5%†	No	Retry mechanism
Hypothesis generation	**Yes**	**6%**	No	Only LLM function with no alternative

*Fallback to `build_domain_driven_plan()` on failure.

†3 of 8 successful steps were rescued by retry.

the 7B bottleneck is *analytical reasoning in code*, not data format knowledge. Creative hypothesis generation—the only function that is both LLM-dependent and without deterministic alternative—achieves only 6% execution success, motivating future work with larger local models or retrieval-augmented generation.

6 Limitations

Several limitations constrain the current system.

The 7B LLM achieves only 54% success on predefined analysis tasks and 6% on novel investigations. Complex analyses (compositional regression, network inference) frequently fail even after three retries. The pure-Python statistical implementations use normal and χ^2 approximations that are less accurate than exact tests for small samples ($n < 20$); scipy-backed tests should be preferred in production. Decision tree thresholds ($p < 0.05$, $d > 0.5$, fc > 5) are drawn from literature conventions, not optimized for specific datasets; different research contexts may warrant different thresholds. The eight-axis weights are hand-selected and not learned. The system currently supports only 16S rRNA amplicon data. The evaluation covers four datasets and seven scenarios; a comprehensive validation would require additional experimental designs

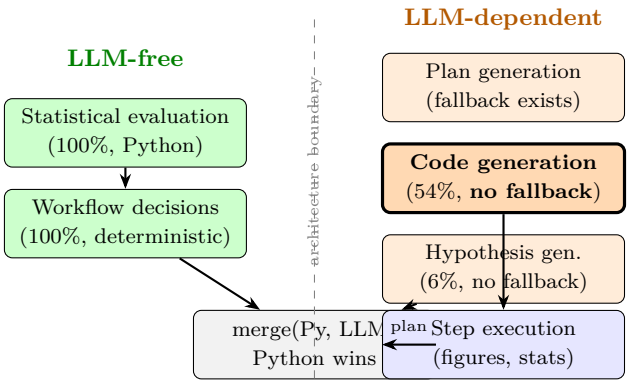

Figure 8: Functional decomposition by LLM dependency. Left (green): functions handled entirely by deterministic Python code—no LLM involvement. Right (orange): LLM-dependent functions; the bold box (code generation) is the only component with no deterministic fallback. The dashed line represents the architectural boundary: decisions flow left-to-right but Python always overrides LLM on conflict.

(longitudinal, case-control with confounders) and comparison with independent human experts rather than a literature-derived protocol.

7 Discussion

The central finding of this work is that the majority of analytical decisions in a microbiome workflow can be made deterministically from data, without LLM involvement. Of the three functional categories identified in Section 5.8, only creative hypothesis generation requires a language model, and even that function is currently unreliable at 7B scale. This suggests that the path to autonomous scientific analysis is not primarily through larger language models, but through better formalization of domain-specific reasoning as auditable, deterministic rules.

The zero false-negative rate across 147 decisions is a stronger safety guarantee than the accuracy percentage alone conveys. A system that occasionally runs an unnecessary analysis (false positive) wastes computation; a system that skips a necessary analysis (false negative) misses a finding. The DecisionEngine's conservative bias—it keeps steps when uncertain—aligns with the scientific principle of not prematurely discarding potentially informative analyses.

The accuracy floor of 52% on taxonomy-absent datasets highlights that adaptive decision-making is bounded by data completeness. This is not a limitation of the decision logic but of the input: without genus-level abundances, differential-abundance-dependent decisions cannot be made. This motivates future work on *data-acquisition decisions*— the system could recommend running additional upstream analyses (e.g., taxonomy classification) when it detects that missing data features limit its decision accuracy.

The LLM performance data (54% registry, 6% investigation) provides a quantitative baseline for future model comparisons. As local models scale from 7B to 30B+ parameters, we expect Category 2 (code generation) success to increase substantially, while Category 3 (hypothesis generation) may require fundamentally different approaches such as retrieval-augmented generation or tool-use frameworks.

seq2pipe System Evaluation Summary

Component	Metric	Value	Assessment
DataInspector	scipy concordance (U/H stat)	Exact match	VALIDATED
	scipy concordance (p-value)	MAE=0.02, max=0.09	Acceptable
	Significance agreement	99-100%	VALIDATED
	Biological conclusions (Caporaso 2011)	7/7 confirmed	VALIDATED
DecisionEngine	Expert agreement (best case)	95%	Strong
	Expert agreement (mean, 7 scenarios)	66.7%	Moderate
	Wrong skips (false negatives)	0 / 147 decisions	SAFE
	Noise reduction (best case)	87.5%	Effective
	Reproducibility (30 runs)	100%	DETERMINISTIC
LLM (7B)	Registry step success	54% (7/13)	Marginal
	Investigation success	6% (1/17)	Inadequate
	Retry rescue rate	37.5% of successes	Partial
	Dominant error	NameError (18/30)	Fixable
	Template improvement	Import errors eliminated	Partial fix
Overall	Deterministic decisions	100% LLM-free	Core strength
	LLM-failure tolerance	System completes	Robust
	Bottleneck	7B code generation	Future work

Figure 9: System evaluation summary. Green = validated or safe; red = inadequate. The core contribution—deterministic, LLM-free decision-making—is the system's primary strength.

8 Conclusion

We presented SEQ2PIPE, a closed-loop system for adaptive microbiome analysis that architecturally separates *what to analyze* (deterministic, LLM-free) from *how to execute the analysis* (LLM-assisted code generation). Evaluation on seven scenarios from four public datasets demonstrated that the deterministic DecisionEngine achieves up to 95% expert agreement with zero false-negative skips and absolute reproducibility. The LLM remains essential for code generation (54% success on predefined tasks) and is the system's primary bottleneck: creative investigations succeed at only 6%, and the dominant error modes (`NameError`, `KeyError`) reflect the 7B model's limited capacity for code synthesis rather than reasoning.

The core design principle is that **workflow decisions should be grounded in data, not in language model outputs**, while acknowledging that **analysis execution currently requires the LLM**. This separation yields a system where decision correctness can be validated independently of code generation quality—an architectural property that enables targeted improvement of either component without affecting the other. As local LLMs scale to 30B+ parameters and template-based code generation matures, we expect the execution bottleneck to narrow, bringing the system closer to fully autonomous operation.

References

Anderson, M. J. (2001). A new method for non-parametric multivariate analysis of variance. *Austral Ecology*, 26(1), 32–46.

Bolyen, E., et al. (2019). Reproducible, interactive, scalable and extensible microbiome data science using QIIME 2. *Nature Biotechnology*, 37(8), 852–857.

Callahan, B. J., et al. (2016). DADA2: High-resolution sample inference from Illumina amplicon data. *Nature Methods*, 13(7), 581–583.

Di Tommaso, P., et al. (2017). Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35(4), 316–319.

Feurer, M., et al. (2015). Efficient and robust automated machine learning. *NeurIPS*, 28.

Gloor, G. B., et al. (2017). Microbiome datasets are compositional: and this is not optional. *Frontiers in Microbiology*, 8, 2224.

Hong, S., et al. (2024). Data Interpreter: An LLM Agent For Data Science. *arXiv:2402.18679*.

Köster, J. & Rahmann, S. (2012). Snakemake—a scalable bioinformatics workflow engine. *Bioinformatics*, 28(19), 2520–2522.

McMurdie, P. J. & Holmes, S. (2014). Waste not, want not: why rarefying microbiome data is inadmissible. *PLOS Computational Biology*, 10(4), e1003531.

Thornton, C., et al. (2013). Auto-WEKA: combined selection and hyperparameter optimization. *KDD '13*, 847–855.

Wasserstein, R. L., et al. (2019). Moving to a world beyond "$p < 0.05$". *The American Statistician*, 73(sup1), 1–19.

Willis, A. D. (2019). Rarefaction, alpha diversity, and statistics. *Frontiers in Microbiology*, 10, 2407.

Wilson, E. B. & Hilferty, M. M. (1931). The distribution of chi-square. *PNAS*, 17(12), 684–688.